Practical No. 1

Aim: Document Indexing and Retrieval

A) Implement an inverted index construction algorithm.
B) Build a simple document retrieval system using the constructed index.

INPUT:

```
import re
from collections import defaultdict

def tokenize(text):
    text = text.lower()
    tokens=re.findall(r'\b[a-z0-9]+\b', text)
    return tokens

def build_inverted_index(documents):
    index = defaultdict(set)

    for doc_id, text in documents.items():
        tokens = tokenize(text)
        for token in tokens:
            index[token].add(doc_id)

    return index

def search_or(index, query):
    result = set()
    tokens = tokenize(query)
    for token in tokens:
        result = index.get(token, set())
    return result

def search_and(index, query):
    tokens = tokenize(query)
    if not tokens:
        return set()

    result = index.get(tokens[0], set()).copy()
    for token in tokens[i:]:
        result &= index.get(token, set())
    return result

if __name__== "__main__":
    documents = {
        1: "Apple banana fruit recipe.",
```

```
    2: " Banana smoothie is a healthy drink.",
    3: " I prefer apple pie over any other fruit dessert.",
    4: "Fruit salad is delicious and easy to make."
}

inverted_index = build_inverted_index(documents)

print("=== Inverted Index ===")
for term, postings in inverted_index.items():
    print(f"{term}: {sorted(postings)}")

print("\n=== Search Results ===")
query1 = "appple drink"
query2 = "banana drink"

print(f"OR search for '{query1}':", search_or(inverted_index, query1))
print(f"AND search for '{query1}':", search_or(inverted_index, query1))

print(f"OR search for '{query2}':", search_or(inverted_index, query1))
print(f"AND search for '{query2}':", search_or(inverted_index, query1))
```

OUTPUT:

```
=== Inverted Index ===
apple: [1, 3]
banana: [1, 2]
fruit: [1, 3, 4]
recipe: [1]
smoothie: [2]
is: [2, 4]
a: [2]
healthy: [2]
drink: [2]
i: [3]
prefer: [3]
pie: [3]
over: [3]
any: [3]
other: [3]
dessert: [3]
salad: [4]
delicious: [4]
and: [4]
easy: [4]
to: [4]
make: [4]

=== Search Results ===
OR search for 'appple drink': {2}
AND search for 'appple drink': {2}
OR search for 'banana drink': {2}
AND search for 'banana drink': {2}
```

Practical No. 2

Aim: Retrieval Models

A) Implement the boolean retrieval model and process queries.
B) Implement the vector space model with TF-IDF weighting and cosine similarity.

INPUT:

```
import re
documents = {
    1: "apple banana mango",
    2: "banana orange apple",
    3: "grapes mango orange",
    4: "banana strawberry",
}
def build_inverted_index(docs):
    inverted = {}
    for doc_id, text in docs.items():
        terms = text.lower().split()
        for term in terms:
            if term not in inverted:
                inverted[term] = set()
            inverted[term].add(doc_id)
    return inverted

inverted_index = build_inverted_index(documents)
UNIVERSAL_SET = set(documents.keys())

def AND(a,b):
    return a & b

def OR(a,b):
    return a | b

def NOT(a):
    return UNIVERSAL_SET - a

def tokenize(query):
    return re.findall(r'\w+|[()]',query)

precedence = {"NOT": 3, "AND": 2, "OR": 1}

def infix_to_postfix(tokens):
    output = []
    stack = []
```

```
    for token in tokens:
        tok = token.upper()
        if tok in precedence:
            while stack and stack[-1] in precedence and precedence[stack[-1]] >=
precedence[tok]:
                output.append(stack.pop())
            stack.append(tok)
        elif token == "(":
            stack.append(token)
        elif token == ")":
            while stack and stack[-1] != "(":
                output.append(stack.pop())
            stack.pop()

        else:
            output.append(token)
    while stack:
        output.append(stack.pop())
    return output

def evaluate_postfix(postfix):
    stack = []
    for token in postfix:
        tok = token.upper()
        if tok == "AND":
            b = stack.pop()
            a = stack.pop()
            stack.append(AND(a,b))
        elif tok == "OR":
            b = stack.pop()
            a = stack.pop()
            stack.append(OR(a,b))
        elif tok == "NOT":
            a = stack.pop()
            stack.append(NOT(a))
        else:
            stack.append(inverted_index.get(token.lower(), set()))
    return stack.pop()

def boolean_query(query):
    tokens = tokenize(query)
    postfix = infix_to_postfix(tokens)
    print("Tokens: ", tokens)
    print("Postfix: ", postfix)
```

```
    result = evaluate_postfix(postfix)
    return result

queries = [
    "apple AND banana",
    "mango OR strawberry",
    "banana and NOT apple",
    "(apple AND banana) OR mango",
    "NOT orange"
]
for q in queries:
    print("\nQuery:", q)
    print("Result:", boolean_query(q))
```

OUTPUT:

```
Query: apple AND banana
Tokens:  ['apple', 'AND', 'banana']
Postfix:  ['apple', 'banana', 'AND']
Result: {1, 2}

Query: mango OR strawberry
Tokens:  ['mango', 'OR', 'strawberry']
Postfix:  ['mango', 'strawberry', 'OR']
Result: {1, 3, 4}

Query: banana and NOT apple
Tokens:  ['banana', 'and', 'NOT', 'apple']
Postfix:  ['banana', 'apple', 'NOT', 'AND']
Result: {4}

Query: (apple AND banana) OR mango
Tokens:  ['(', 'apple', 'AND', 'banana', ')', 'OR', 'mango']
Postfix:  ['apple', 'banana', 'AND', 'mango', 'OR']
Result: {1, 2, 3}

Query: NOT orange
Tokens:  ['NOT', 'orange']
Postfix:  ['orange', 'NOT']
Result: {1, 4}
```

Practical No. 3

Aim: Spelling correction in IR System.

A) Develop a spelling correction module using edit distance algorithms.

B) Integrate the spelling correction module into an information retrieval system.

INPUT:

```
def edit_distance(a,b):
    m,n=len(a), len(b)
    dp = [[0]*(n+1) for _ in range(m+1)]

    for i in range(m+1):
        dp[i][0]=i
    for j in range(n+1):
        dp[0][j]=j



    for i in range (1, m+1):
        for j in range(1, n+1):
            cost = 0 if a[i-1] == b[j-1] else 1
            dp[i][j] = min(
                dp[i-1][j] + 1,
                dp[i][j-1] + 1,
                dp[i-1][j-1] +cost
                )
    return dp[m][n]

def correct(word, vocab):
    best_word = word
    best_dist = 999
    for v in vocab:
        d = edit_distance(word, v)
        if d < best_dist:
            best_dist = d
            best_word = v
    return best_word

vocab = ["Information",  "retrieval", "spelling", "correction", "module"]
query = "speling"
print("Original:",  query)
print("Corrected:", correct(query, vocab))
```

OUTPUT:

```
Original: speling
Corrected: spelling
```

Practical No. 4

Aim: Evaluation Matrices for IR System

A) Calculate precision, recall, and F-measure for a given set of retrieval results.

B) Use an evaluation toolkit to measure average precision and other evaluation metrics.

a) Precision

Precision measures how many retrieved documents are relevant.

$$\text{Precision} = \frac{\text{Relevant Documents Retrieved}}{\text{Total Documents Retrieved}}$$

b) Recall

Recall measures how many relevant documents were successfully retrieved.

$$\text{Recall} = \frac{\text{Relevant Documents Retrieved}}{\text{Total Relevant Document}}$$

c) F-measure(F1-score)

The F-measure combines precision and recall using the harmonic mean.

$$F1 = \frac{2 * (\text{Precision} * \text{Recall})}{\text{Precision} + \text{Recall}}$$

Step 1: Precision

$$\text{Precision} = \frac{6}{10} = 0.6$$

Step 2: Recall

$$\text{Recall} = \frac{6}{8} = 0.75$$

Step 3: F-measure

$$F1 = \frac{2 * 0.6 * 0.75}{0.6 + 0.75} = \frac{0.90}{1.35} = 0.667$$

INPUT:

```
def evaluation_metrics(retrieved, relevant_retrieved, total_relevant):
    precision = relevant_retrieved/retrieved
    recall = relevant_retrieved/total_relevant
    f_measure = (2 * precision * recall)/(precision + recall)
    return precision, recall, f_measure

retrieved = int(input("Enter total number of documentd retrieved: "))
relevant_retrieved  = int(input("Enter number of relevant documents retrieved: "))
total_relevant = int(input("Enter total number of relevant documents: "))

precision = relevant_retrieved/retrieved
recall = relevant_retrieved/total_relevant

f_measure = (2 * precision * recall)/(precision + recall)

print("\nEvaluation Metrics:")
print("Precision=", round(precision, 3))
print("Recall=", round(recall, 3))
print("F-Measure", round(f_measure, 3))
```

OUTPUT:

```
Enter total number of documentd retrieved: 10
Enter number of relevant documents retrieved: 6
Enter total number of relevant documents: 8

Evaluation Metrics:
Precision= 0.6
Recall= 0.75
F-Measure 0.667
```

INPUT:

```
print("------------ Sample Data ---------------")
print("-------------Retrieval Documents (ranked) --------------")
retrieved_docs = ["D1", "D2", "D3", "D4", "D5"]
print(retrieved_docs)

print("--------Relevant documents-------")
relevant_docs = {"D1", "D3", "D6"}
print(relevant_docs)

results = {
    "Q1": ["D1", "D2", "D3", "D4"],
    "Q2": ["D2", "D3", "D5"]
}

relevance = {
    "Q1": {"D1", "D2"},
    "Q2": {"D3", "D4"}
}

def precision(retrieved, relevant):
    retrieved_set = set(retrieved)
    rel_ret = retrieved_set.intersection(relevant)
    return len(rel_ret) / len(retrieved) if retrieved else 0

def recall(retrieved, relevant):
    retrieved_set = set(retrieved)
    rel_ret = retrieved_set.intersection(relevant)
    return len(rel_ret) / len(relevant) if relevant else 0

def f_measure(p, r):
    return (2 * p * r) / (p + r) if (p + r) else 0

def average_precision(retrieved, relevant):
    score = 0.0
    rel_count = 0

    for i, doc in enumerate(retrieved, start=1):
        if doc in relevant:
            rel_count += 1
            score += rel_count / i

    return score / len(relevant) if relevant else 0
```

```
def mean_average_precision(results, relevance):
    total_ap = 0

    for q in results:
        total_ap += average_precision(results[q], relevance[q])

    return total_ap / len(results)

p = precision(retrieved_docs, relevant_docs)
r = recall(retrieved_docs, relevant_docs)
f1 = f_measure(p, r)
ap = average_precision(retrieved_docs, relevant_docs)

print("\nSingle Query Evaluation")
print("--------------------------------")
print("Precision :", round(p, 2))
print("Recall :", round(r, 2))
print("F-Measure :", round(f1, 2))
print("Avg Precision (AP):", round(ap, 2))

map_score = mean_average_precision(results, relevance)

print("\nMultiple Query Evaluation")
print("--------------------------------------")
print("Mean Average Precision (MAP):", round(map_score, 2))
```

OUTPUT:

```
------------ Sample Data ---------------
-------------Retrieval Documents (ranked) --------------
['D1', 'D2', 'D3', 'D4', 'D5']
--------Relevant documents-------
{'D6', 'D3', 'D1'}

Single Query Evaluation
--------------------------------
Precision : 0.4
Recall : 0.67
F-Measure : 0.5
Avg Precision (AP): 0.56

Multiple Query Evaluation
--------------------------------------
Mean Average Precision (MAP): 0.62
```

Practical No. 5

Aim: Text Categorization

A) Implement a text classification algorithm (e.g., Naive Bayes or Support Vector Machines).
B) Train the classifier on a labelled dataset and evaluate its performance.

INPUT:

```
# Text Categorization using Support Vector machine (SVM)

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


data = {
    "text": [
        "India won the cricket match",
        "The government passed a new law",
        "Football team won the championship",
        "Election results announced",
        "The player scored a century",
        "Parliament session begins",
        "Team lost the match",
        "Minister gave a speech",
        "World Cup final match",
        "New tax policy introduced"
    ],
    "label": [
        "Sport","Politics","Sport","Politics","Sport",
        "Politics","Sport","Politics","Sport","Politics"
    ]
}

df = pd.DataFrame(data)


X_train, X_test, y_train, y_test = train_test_split(
    df["text"], df["label"], test_size=0.3, random_state=1
)

tfidf = TfidfVectorizer(stop_words='english')
```

```
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)


svm_model = LinearSVC()
svm_model.fit(X_train_tfidf, y_train)


y_pred = svm_model.predict(X_test_tfidf)


print("Accuracy :", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix :\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report :\n", classification_report(y_test, y_pred))


sample = [" India won the cricket match"]
sample_vec = tfidf.transform(sample)
print("\nPrediction:", svm_model.predict(sample_vec)[0])
```

OUTPUT:

```
Accuracy : 1.0

Confusion Matrix :
 [[1 0]
 [0 2]]

Classification Report :
              precision    recall  f1-score   support

    Politics       1.00      1.00      1.00         1
       Sport       1.00      1.00      1.00         2

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3


Prediction: Sport
```

Practical No. 6

Aim: Clustering for Information Retrieval

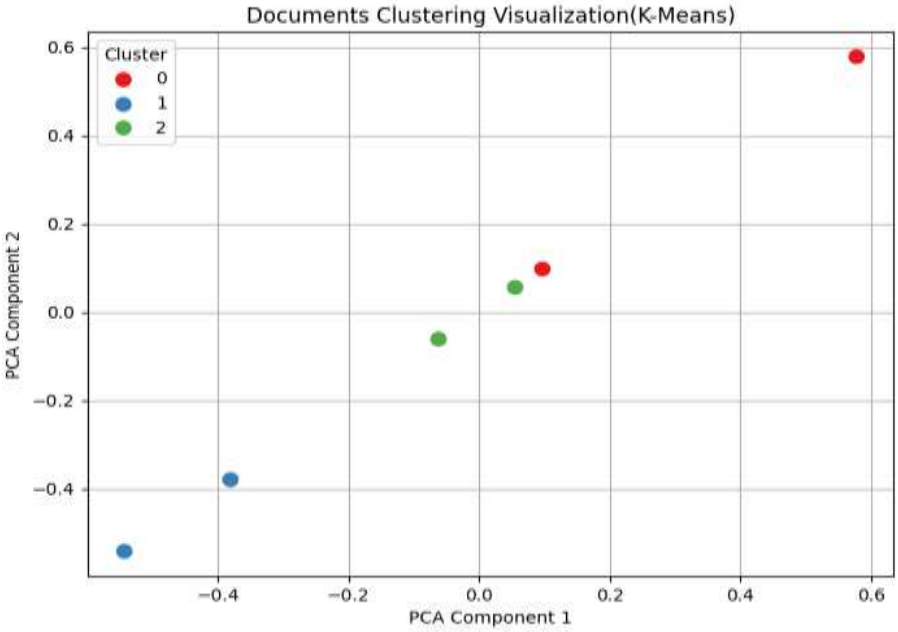
A) Implement a clustering algorithm (e.g., K-means or hierarchical clustering).
B) Apply the clustering algorithm to a set of documents and evaluate the clustering results.

INPUT:

```
#Documents Clustering for Information Retrieval using K-Means

#Step 1: Import Required Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA


documents = [
    "Machine learning is used for data analysis",
    "Artificial intelligence and machine learninfg",
    "Deep learning advances AI",
    "Football is a popular sport",
    "Cricket and football are outdoor games",
    "sports players train regularly",
    "Python is a programming language",
    "Java and Python are used in software development",
    "Programming help build applications"
    ]

vectorizer = TfidfVectorizer(
    stop_words='english',
    lowercase=True
)
X=vectorizer.fit_transform(documents)

k=3

kmeans = KMeans(
    n_clusters=k,
```

```
    random_state=42,
    n_init=10
)

kmeans.fit(X)
labels = kmeans.labels_

df = pd.DataFrame({
    "Document": documents,
    "cluster": labels
    })

print("\n=====Documents Clustering Result ====")
print(df)
sil_score = silhouette_score(X,labels)
print("\nSilhouette Score:", sil_score)

terms = vectorizer.get_feature_names_out()
for i in range(k):
    print(f"science {i}:")
    center_terms = kmeans.cluster_centers_[i]
    top_terms = center_terms.argsort()[-5:][::-1]
    for term in top_terms:
        print(terms[term])

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X.toarray())

plt.figure(figsize = (8,6))
sns.scatterplot(
    x = X_pca[:, 0],
    y = X_pca[:, 0],
    hue=labels,
    palette="Set1",
     s=100
    )

plt.title("Documents Clustering Visualization(K-Means)")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.legend(title="Cluster")
plt.grid(True)
plt.show()
```

OUTPUT:




```
=====Documents Clustering Result ====
                                          Document  cluster
0        Machine learning is used for data analysis        2
1     Artificial intelligence and machine learninfg        2
2                        Deep learning advances AI        2
3                      Football is a popular sport        0
4            Cricket and football are outdoor games        0
5                    sports players train regularly        0
6                  Python is a programming language        1
7  Java and Python are used in software development        1
8               Programming help build applications        1

Silhouette Score: 0.0553015159177245
science 0:
football
sport
popular
outdoor
games
science 1:
programming
python
language
build
help
science 2:
machine
learning
learninfg
advances
deep
```

Practical No. 7

Aim: Indexing and Crawling.

A) Develop a web crawler to fetch and index web pages.
B) Handle challenges such as robots.txt, dynamic content, and crawling delays.

INPUT:

```
import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin, urlparse
import time

visited = set()

def crawl(url, domain, delay =0.1):
    if url in visited:
        return 0

    print(f"crawling {url}")
    visited.add(url)

    try:
        response = requests.get(url, timeout=5)
        if response.status_code != 200:
            return 0
    except requests.RequestException:
            return 0

    soup = BeautifulSoup(response.text, "html.parser")
    links = soup.find_all("a", href=True)

    link_count = 0

    for link in links:
        absolute_url = urljoin(url, link["href"])
        parsed = urlparse(absolute_url)

        if parsed.netloc == domain:
            link_count += 1
            if absolute_url not in visited:
                time.sleep(delay)
                crawl(absolute_url, domain, delay)
    return link_count

def start_crawler(start_url):
```

```
    domain = urlparse(start_url).netloc
    total_links = crawl(start_url, domain)
    print("\n==== CRAWLING COMPLETE =====")
    print(f"Total pages crawled: { len(visited) } ")
    print(f"Total links found: { total_links } ")

if __name__ == "__main__":
    website = input("Enter website URL (e.g., https://example.com): ")
    start_crawler(website)
```

OUTPUT:

```
Enter website URL (e.g., https://example.com): https://zenoxit.com/
crawling https://zenoxit.com/
crawling https://zenoxit.com/software-for-institute-management.html
crawling https://zenoxit.com/software-for-institute-management.html#.
crawling https://zenoxit.com/images/graphics/cmis/Enquiry.jpg
crawling https://zenoxit.com/images/graphics/cmis/singlehalltickettask.jpg
crawling https://zenoxit.com/images/graphics/cmis/Voucher.jpg
crawling https://zenoxit.com/images/graphics/cmis/questionpaper.jpg
crawling https://zenoxit.com/images/graphics/cmis/idcard-rpt.jpg
crawling https://zenoxit.com/images/graphics/cmis/installmentreceivable-rpt.jpg
crawling https://zenoxit.com/images/graphics/cmis/Company.jpg
crawling https://zenoxit.com/images/graphics/cmis/AddCourse.jpg
crawling https://zenoxit.com/ajax-work/CMIS-portfolio.html
crawling https://zenoxit.com/ajax-work/CMIS-portfolio.html#.
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/Addsource.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/HardwareDetails.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/questionbank.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/JobDesignation.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rptbatch.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/otherincome.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/InterviewSchedule.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/practicalexammarks.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rptstudentattendance.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/bulkquestionbank.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/EmployeeSalary.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/JobOpening.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/hallticketbulkupload.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/MaterialIssue.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/batchschedule.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/leavemanagement.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rptenquirysource.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/FeesReceipt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/AddEnq.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/examschedule.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/MaterialPurchase.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/EmployeeAttendance.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/Staffattendance.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/StaffPerformance.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/Admission.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/FeesFollowup.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rptbookissue.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/profitnloss_cmis.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/feessummary_cmis.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rptanswersheet.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/EnquiryFollowup.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/Stattendance.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rptjobopening.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rptmaterialstock.jpg
```

```
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rpthalltickets.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rptsales.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rptfeedback.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rpthardware.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rpttax.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rptcourse.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rptfeesfollowup.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rptinterviewshedule.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/admissionaddress-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/admissionbirthday-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/allsummary-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/answaersheet-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/certificate-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/enquiryabirthday-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/enquiryaddress-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/enquiryaremark-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/enquiryarfollowup-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/enquirysummary-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/feeshistory-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/feesreceivable-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/feesreceived-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/feessummary-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/hallticket-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/HR-rpt.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rptemployee.jpg
crawling https://zenoxit.com/ajax-work/images/graphics/cmis/rptfeesreceivable.jpg
crawling https://zenoxit.com/software-for-institute-management.html#collapseOne
crawling https://zenoxit.com/software-for-institute-management.html#collapseTwo
crawling https://zenoxit.com/software-for-institute-management.html#collapseThree
crawling https://zenoxit.com/software-for-institute-management.html#collapsefour
crawling https://zenoxit.com/software-for-institute-management.html#collapsefiv
crawling https://zenoxit.com/software-for-institute-management.html#collapsesix
crawling https://zenoxit.com/software-for-institute-management.html#collapsesev
crawling https://zenoxit.com/Download/DEMOSetup/CMIS/CMIS_7_0.zip
crawling https://zenoxit.com/Download/DEMOSetup/CMIS/CMIS_6_0.zip
crawling https://zenoxit.com/bulk-sms-service-india.html
crawling https://zenoxit.com/bulk-sms-service-india.html#collapseOne
crawling https://zenoxit.com/bulk-sms-service-india.html#collapseTwo
crawling https://zenoxit.com/bulk-sms-service-india.html#collapseThree
crawling https://zenoxit.com/bulk-sms-service-india.html#collapsefour
crawling https://zenoxit.com/bulk-sms-service-india.html#collapsefiv
crawling https://zenoxit.com/bulk-sms-service-india.html#collapsesix
crawling https://zenoxit.com/#.
crawling https://zenoxit.com/Contact.html#content
crawling https://zenoxit.com/Contact.html

==== CRAWLING COMPLETE =====
Total pages crawled: 92
Total links found: 7
```

Practical No. 8

Aim: Link Analysis and Page Rank

A) Implement the PageRank algorithm to rank web pages based on link analysis.
B) Apply the PageRank algorithm to a small web graph and analyze the results.

INPUT;

```
pages = ['A', 'B', 'C', 'D']
# Links structure
links = {
   'A' : ['B', 'C'],
   'B' : ['C'],
   'C' : ['A'],
   'D' : ['C']
}
# Initial PageRank
pagerank = {}
N = len(pages)
for page in pages:
   pagerank[page] = 1/N
d = 0.85 # damping factor
iterations = 10
# PageRank Calculation
for i in range(iterations):
   new_pagerank = {}
   for page in pages:
      rank = (1 - d) / N
      for other_page in pages:
         if page in links.get(other_page, []):
            rank += d *  (pagerank[other_page] / len(links[other_page]))
      new_pagerank[page] = rank
   pagerank = new_pagerank
# Display Result
print("Final PageRank Values: ")
for page, rank in pagerank.items():
   print(page, ":", round(rank, 4))
```

Output :

```
Final PageRank Values:
A :
 0.3751
B : 0.1949
C : 0.3925
D : 0.0375
```

Practical No. 9

Aim: Learning to Rank

A) Implement a text summarization algorithm (e.g., extractive or abstractive).

B) Build a question-answering system using techniques such as information extraction.

INPUT:

```
import numpy as np
from sklearn.svm import LinearSVC
from sklearn.metrics import ndcg_score

x=np.array([
    [0.2, 0.1],
    [0.4, 0.6],
    [0.1, 0.8],
    [0.7, 0.2],
    [0.9, 0.5],
    [0.3, 0.4]
])
y = np.array([3, 2, 1, 3, 2, 1])
qids = np.array([1, 1, 1, 2, 2, 2])
x_pair = []
y_pair = []
for q in np.unique(qids):
    idx = np.where(qids == q)[0]
    for i in idx:
        for j in idx:
            if y[i] > y[j]:
                x_pair.append(x[i] - x[j])
                y_pair.append(1)
            elif y[i] < y[j]:
                x_pair.append(x[i] - x[j])
                y_pair.append(-1)
x_pair = np.array(x_pair)
y_pair = np.array(y_pair)
model = LinearSVC()
model.fit(x_pair, y_pair)
print("Model Trained Successfully")

scores = model.decision_function(x)
print("Ranking Scores:", scores)

for q in np.unique(qids):
```

```
idx = np.where(qids == q)[0]
true_rel = y[idx].reshape(1, -1)
pred_scores = scores[idx].reshape(1, -1)
ndcg = ndcg_score(true_rel, pred_scores)
print("Query", q, "NDCG:", round(ndcg, 3))
```

OUTPUT:

```
Model Trained Successfully
Ranking Scores: [ 0.04646018 -0.60619469 -1.28761062  0.42477876  0.12168142 -0.
36725664]
Query 1 NDCG: 1.0
Query 2 NDCG: 1.0
```